ProjectWA-EVChargerOpt

August 10, 2026

[7]:
```
# On my Mac: brew install osmium-tools
# for reference from filter command above: road categories (including link␣
 ↪ramps) included in osmium pre-processing filter
# > osmium tags-filter washington-260425.osm.pbf \
# >  w/
 ↪highway=motorway,motorway_link,trunk,trunk_link,primary,primary_link,secondary,secondary_li
 ↪\
# >  -o washington-drivable.osm.pbf

DRIVABLE_ROADS = [
    "motorway", "motorway_link",
    "trunk", "trunk_link",
    "primary", "primary_link",
    "secondary", "secondary_link",
    "tertiary", "tertiary_link",
    # these had to be pre-filtered. The puget sound crashed the kernel.
    # removing them dropped a 71 MB file to a 13 MB file that could be processed
    # "unclassified", "residential",
    # "service"
]
```

[4]:
```
# 2 x 4 Grid covering all of Washington State
wa_regions_8 = {
    # --- North Band (47.25°N to 49.00°N) ---
    "nw_olympic_peninsula": [-124.800, 47.250, -122.825, 49.000],  # Port␣
 ↪Angeles, Strait of Juan de Fuca
    "puget_sound_north":    [-122.825, 47.250, -120.850, 49.000],  # Seattle,␣
 ↪Everett, Bellingham, N. Cascades
    "north_central":        [-120.850, 47.250, -118.875, 49.000],  # Wenatchee,␣
 ↪Chelan, Okanogan
    "ne_spokane":           [-118.875, 47.250, -116.900, 49.000],  # Spokane,␣
 ↪Colville, ID border north

    # --- South Band (45.50°N to 47.25°N) ---
    "sw_pacific_coast":     [-124.800, 45.500, -122.825, 47.250],  # Grays␣
 ↪Harbor, Longview, SW Coast
```

```
    "south_cascades":       [-122.825, 45.500, -120.850, 47.250],  # Tacoma␣
 ↪south, Olympia, Mt Rainier, Vancouver
    "se_tri_cities":        [-120.850, 45.500, -118.875, 47.250],  # Yakima,␣
 ↪Tri-Cities, Columbia Basin
    "se_palouse":           [-118.875, 45.500, -116.900, 47.250]   # Pullman,␣
 ↪Walla Walla, ID border south
}
```

[7]:
```
import os
import time
from concurrent.futures import ProcessPoolExecutor
from region_worker import process_region

if __name__ == "__main__":
    # target 8 workers for the 8 regions
    MAX_WORKERS = 8

    print(f"Spawning ProcessPoolExecutor with {MAX_WORKERS} workers across␣
 ↪{len(wa_regions_8)} regions...\n", flush=True)

    start_time = time.time()

    # Process all 8 bounding boxes simultaneously
    with ProcessPoolExecutor(max_workers=MAX_WORKERS) as executor:
        results = list(executor.map(process_region, wa_regions_8.items()))

    elapsed = time.time() - start_time

    print("\n" + "=" * 55)
    print(f"PARALLEL EXECUTION COMPLETE IN {elapsed:.2f} SECONDS")
    print("=" * 55)
    # Unpack all 5 values returned by region_worker.py
    for region_name, n_nodes, n_edges, node_path, edge_path in results:
        print(f"{region_name:<22} | Nodes: {str(n_nodes):<8} | Edges:␣
 ↪{str(n_edges):<8}")
```

```
Spawning ProcessPoolExecutor with 8 workers across 8 regions…

[SAVED] se_palouse: 48,891 nodes, 49,372 edges ->
./pq_data/se_palouse_edges.parquet
[SAVED] nw_olympic_peninsula: 39,201 nodes, 39,385 edges ->
./pq_data/nw_olympic_peninsula_edges.parquet
[SAVED] north_central: 61,662 nodes, 62,138 edges ->
./pq_data/north_central_edges.parquet
[SAVED] sw_pacific_coast: 82,039 nodes, 83,210 edges ->
./pq_data/sw_pacific_coast_edges.parquet
[SAVED] ne_spokane: 95,479 nodes, 97,215 edges ->
```

```
./pq_data/ne_spokane_edges.parquet
[SAVED] se_tri_cities: 88,272 nodes, 90,381 edges ->
./pq_data/se_tri_cities_edges.parquet
[SAVED] south_cascades: 186,330 nodes, 189,690 edges ->
./pq_data/south_cascades_edges.parquet
[SAVED] puget_sound_north: 338,591 nodes, 347,307 edges ->
./pq_data/puget_sound_north_edges.parquet

============================================================
PARALLEL EXECUTION COMPLETE IN 43.34 SECONDS
============================================================
nw_olympic_peninsula   | Nodes: 39201    | Edges: 39385
puget_sound_north      | Nodes: 338591   | Edges: 347307
north_central          | Nodes: 61662    | Edges: 62138
ne_spokane             | Nodes: 95479    | Edges: 97215
sw_pacific_coast       | Nodes: 82039    | Edges: 83210
south_cascades         | Nodes: 186330   | Edges: 189690
se_tri_cities          | Nodes: 88272    | Edges: 90381
se_palouse             | Nodes: 48891    | Edges: 49372
```

[8]:
```python
import glob
import pandas as pd
import geopandas as gpd
import networkx as nx

OUTPUT_DIR = "./pq_data"

# load all regional node and edge Parquet files
node_files = glob.glob(os.path.join(OUTPUT_DIR, "*_nodes.parquet"))
edge_files = glob.glob(os.path.join(OUTPUT_DIR, "*_edges.parquet"))

print("Loading and deduplicating regional Parquet files...", flush=True)

# Combine and deduplicate by global OSM ID
all_nodes = pd.concat([gpd.read_parquet(f) for f in node_files]).
 ↪drop_duplicates(subset=["id"])
all_edges = pd.concat([gpd.read_parquet(f) for f in edge_files]).
 ↪drop_duplicates(subset=["id"])

# this ensures ID columns are clean integers
all_nodes["id"] = all_nodes["id"].astype(int)
all_edges["u"] = all_edges["u"].astype(int)
all_edges["v"] = all_edges["v"].astype(int)

# Filter out any orphan edges whose endpoint nodes weren't saved
valid_node_ids = set(all_nodes["id"])
all_edges = all_edges[
```

```
    all_edges["u"].isin(valid_node_ids) & all_edges["v"].isin(valid_node_ids)
].copy()

print(f"Total Unique Nodes: {len(all_nodes):,}")
print(f"Total Unique Edges: {len(all_edges):,}")
```

```
Loading and deduplicating regional Parquet files…
Total Unique Nodes: 940,465
Total Unique Edges: 92,666
```

[9]:
```
# assemble the Statewide NetworkX MultiDiGraph (G)
print("Building statewide NetworkX graph object 'G'...", flush=True)
G = nx.MultiDiGraph()

# Add Nodes
for _, row in all_nodes.iterrows():
    G.add_node(
        row["id"],
        x=row.geometry.x,
        y=row.geometry.y,
        geometry=row.geometry
    )

# Add Edges
for _, row in all_edges.iterrows():
    G.add_edge(
        row["u"],
        row["v"],
        key=row.get("key", 0),
        osmid=row["id"],
        length=row.get("length", 0.0),
        highway=row.get("highway", "road"),
        geometry=row.geometry
    )

print("\nSTATEWIDE GRAPH 'G' IS NOW READY IN MEMORY!")
print(f"  - Total Unique Nodes: {G.number_of_nodes():,}")
print(f"  - Total Unique Edges: {G.number_of_edges():,}")
```

```
Building statewide NetworkX graph object 'G'…

STATEWIDE GRAPH 'G' IS NOW READY IN MEMORY!
  - Total Unique Nodes: 940,465
  - Total Unique Edges: 92,666
```

[10]:
```
import pandas as pd
import geopandas as gpd
```

```
import numpy as np
from scipy.spatial import KDTree

# read raw AAA roadside assistance CSV
df_raw = pd.read_csv("AAA_OutofChargeWA_Nov22-Nov23.csv")

# filter down to Out-of-Charge / Low-Fuel EV incidents
ooc_filter = "Out of Fuel / Low Fuel/Out of Charge"
aaa_incidents = df_raw[df_raw["ERS_CATEGORY"] == ooc_filter].copy()

# convert to GeoDataFrame (WGS84 / EPSG:4326)
aaa_incidents = gpd.GeoDataFrame(
    aaa_incidents,
    geometry=gpd.points_from_xy(aaa_incidents["ERS_LONGITUDE"],␣
 ↪aaa_incidents["ERS_LATITUDE"]),
    crs="EPSG:4326"
)

print(f"Loaded {len(aaa_incidents)} out-of-charge EV incidents across␣
 ↪Washington.")
print(aaa_incidents[["TIMESTAMP", "VEH_MAKE", "VEH_MODEL", "ERS_NEAR_CITY"]].
 ↪head())
```

```
Loaded 70 out-of-charge EV incidents across Washington.
   TIMESTAMP VEH_MAKE VEH_MODEL ERS_NEAR_CITY
0   20231104   NISSAN      LEAF       Spokane
1   20230723    TESLA         3  Kettle Falls
2   20221120    TESLA         S        Curlew
3   20231014    TESLA         X         Pasco
4   20231128      BMW        I3      Richland
```

[14]:
```
def snap_gdf_to_graph(gdf, graph):
    """
    Snaps a GeoDataFrame of point geometries to the nearest node ID
    in a NetworkX graph using a KDTree spatial index.
    """
    # Extract node coordinates (lon=x, lat=y) from NetworkX graph
    nodes_data = [(node, data['x'], data['y']) for node, data in graph.
 ↪nodes(data=True)]
    node_ids, lons, lats = zip(*nodes_data)

    # Build 2D spatial index
    tree = KDTree(np.column_stack([lons, lats]))

    # Query nearest node index for all incident points simultaneously
    coords = np.column_stack([gdf.geometry.x, gdf.geometry.y])
    distances, indices = tree.query(coords)
```

```
    # Return matched node IDs and snapping offset distance
    nearest_nodes = [node_ids[i] for i in indices]
    return nearest_nodes, distances
```

[12]:
```
from shapely.geometry import box

# Create GeoDataFrame of your 8 bounding boxes
region_boxes = []
for r_name, r_bbox in wa_regions_8.items():
    region_boxes.append({"region": r_name, "geometry": box(*r_bbox)})

wa_grid_gdf = gpd.GeoDataFrame(region_boxes, crs="EPSG:4326")

# Spatial join to map each incident to its region
incidents_with_region = gpd.sjoin(aaa_incidents, wa_grid_gdf, how="left",␣
 ↪predicate="within")

print("\nOut-of-Charge Incidents per Region:")
print(incidents_with_region["region"].value_counts())
```

```
Out-of-Charge Incidents per Region:
region
puget_sound_north       41
south_cascades          11
sw_pacific_coast         6
ne_spokane               3
se_tri_cities            3
north_central            3
nw_olympic_peninsula     3
Name: count, dtype: int64
```

[13]:
```
# load Existing Infrastructure GeoJSON
try:
    existing_chargers = gpd.read_file("existing_chargers_2023.geojson").
 ↪to_crs(epsg=4326)
except Exception:
    existing_chargers = gpd.GeoDataFrame(columns=["geometry"], crs="EPSG:4326")

print(f"  - Loaded {len(existing_chargers)} existing charger locations.")
```

```
  - Loaded 2048 existing charger locations.
```

[17]:
```
import time
import importlib
```

```
# force reload of updated region_optimizer file (avoid caching)
import region_optimizer
importlib.reload(region_optimizer)
from region_optimizer import optimize_region_pulp

# snap & clean data
print("Pre-snapping and cleaning incidents and existing chargers...",␣
 ↪flush=True)

# snap the 70 incidents to the statewide unified graph
aaa_incidents["nearest_node"], aaa_incidents["snap_dist_deg"] =␣
 ↪snap_gdf_to_graph(aaa_incidents, G)
existing_chargers["nearest_node"], _ = snap_gdf_to_graph(existing_chargers, G)

clean_incidents = aaa_incidents.dropna(subset=["nearest_node"]).copy()
clean_incidents["nearest_node"] = clean_incidents["nearest_node"].astype(int)

clean_chargers = existing_chargers.dropna(subset=["nearest_node"]).copy()
clean_chargers["nearest_node"] = clean_chargers["nearest_node"].astype(int)

# number of EV charger sites to be distributed evenly in 8 WA regions
DEFAULT_QUOTA_PER_REGION = 2

# this is ready for parallel execution once we have access
worker_tasks = [
    (region_name, bbox, clean_incidents, clean_chargers,␣
 ↪DEFAULT_QUOTA_PER_REGION)
    for region_name, bbox in wa_regions_8.items()
]

print(f"\nRunning sequential optimization across {len(worker_tasks)} regions...
 ↪")
start_time = time.time()
# sequential execution loop (memory-safe to avoid crashing kernel envs on␣
 ↪I-GUIDE)
results = []
for task in worker_tasks:
    region_name = task[0]
    try:
        # executes each region cleanly in memory one-by-one (~0.3s each)
        res = optimize_region_pulp(task)
        results.append(res)
    except Exception as e:
        print(f"Warning: Skipped {region_name} due to error: {e}")
        results.append((region_name, []))

elapsed = time.time() - start_time
```

```python
# final summary & node extraction
print("\n" + "=" * 60)
print(f"OPTIMIZATION COMPLETE IN {elapsed:.2f} SECONDS")
print("=" * 60)

all_selected_node_ids = []

for region_name, sites in results:
    num_sites = len(sites)
    all_selected_node_ids.extend(sites)
    print(f"{region_name:<22} | Selected Sites: {num_sites} | Node IDs:␣
 ↪{sites}")

print(f"\nTotal Optimal Charger Sites Selected Statewide:␣
 ↪{len(all_selected_node_ids)}")
```

```
Pre-snapping and cleaning incidents and existing chargers…

Running sequential optimization across 8 regions…
--> [START] Optimization for nw_olympic_peninsula (Quota K=2)…
<-- [DONE] nw_olympic_peninsula: Selected 2 optimal sites.
--> [START] Optimization for puget_sound_north (Quota K=2)…
<-- [DONE] puget_sound_north: Selected 2 optimal sites.
--> [START] Optimization for north_central (Quota K=2)…
<-- [DONE] north_central: Selected 2 optimal sites.
--> [START] Optimization for ne_spokane (Quota K=2)…
<-- [DONE] ne_spokane: Selected 2 optimal sites.
--> [START] Optimization for sw_pacific_coast (Quota K=2)…
<-- [DONE] sw_pacific_coast: Selected 2 optimal sites.
--> [START] Optimization for south_cascades (Quota K=2)…
<-- [DONE] south_cascades: Selected 2 optimal sites.
--> [START] Optimization for se_tri_cities (Quota K=2)…
<-- [DONE] se_tri_cities: Selected 2 optimal sites.
--> [START] Optimization for se_palouse (Quota K=2)…
    Notice: Zero incidents in se_palouse.

============================================================
OPTIMIZATION COMPLETE IN 614.56 SECONDS
============================================================
nw_olympic_peninsula   | Selected Sites: 2 | Node IDs: [5494450248, 47189579]
puget_sound_north      | Selected Sites: 2 | Node IDs: [2927137481, 1315578478]
north_central          | Selected Sites: 2 | Node IDs: [50051615, 47066567]
ne_spokane             | Selected Sites: 2 | Node IDs: [47973570, 47895913]
sw_pacific_coast       | Selected Sites: 2 | Node IDs: [51408922, 49928916]
south_cascades         | Selected Sites: 2 | Node IDs: [110397230, 369930272]
se_tri_cities          | Selected Sites: 2 | Node IDs: [47965994, 48189294]
```

```
se_palouse              | Selected Sites: 0 | Node IDs: []

Total Optimal Charger Sites Selected Statewide: 14
```

[18]:
```python
import pandas as pd
import geopandas as gpd
import folium

# 1. Dynamically extract all selected node IDs from the execution results
all_selected_node_ids = []

for region_name, sites in results:
    num_sites = len(sites)
    all_selected_node_ids.extend(sites)
    print(f"{region_name:<22} | Selected Sites: {num_sites} | Node IDs: {sites}")

print(f"\nTotal Optimal Charger Sites Selected Statewide: {len(all_selected_node_ids)}")

# 2. Dynamically filter all_nodes GeoDataFrame using the extracted ID list
final_sites_gdf = all_nodes[all_nodes["id"].isin(all_selected_node_ids)].copy()
final_sites_gdf["latitude"] = final_sites_gdf.geometry.y
final_sites_gdf["longitude"] = final_sites_gdf.geometry.x

# 3. Export CSV for your report tables/appendices
final_sites_gdf[["id", "latitude", "longitude"]].to_csv("optimal_ev_chargers_wa.csv", index=False)
print(f"Saved {len(final_sites_gdf)} optimized charger coordinates to 'optimal_ev_chargers_wa.csv'")

# 4. Render Interactive Folium Map
m = folium.Map(location=[47.25, -120.85], zoom_start=7, tiles="cartodbpositron")

# Plot Existing Chargers (Light Blue)
if len(clean_chargers) > 0:
    for _, row in clean_chargers.iterrows():
        folium.CircleMarker(
            location=[row.geometry.y, row.geometry.x],
            radius=2, color="#3182bd", fill=True, fill_opacity=0.3,
            tooltip="Existing EV Charger"
        ).add_to(m)

# Plot AAA Incidents (Red)
for _, row in clean_incidents.iterrows():
    folium.CircleMarker(
        location=[row.geometry.y, row.geometry.x],
```

```
        radius=4, color="#e41a1c", fill=True, fill_opacity=0.8,
        tooltip=f"AAA Breakdown: {row.get('VEH_MAKE', 'EV')} ({row.
 ↪get('ERS_NEAR_CITY', '')})"
    ).add_to(m)

# Plot Dynamically Sited New Chargers (Green Bolt)
for _, row in final_sites_gdf.iterrows():
    folium.Marker(
        location=[row.geometry.y, row.geometry.x],
        icon=folium.Icon(color="green", icon="bolt", prefix="fa"),
        tooltip=f"OPTIMAL SITE - Node ID: {row['id']}"
    ).add_to(m)

m
```

```
nw_olympic_peninsula   | Selected Sites: 2 | Node IDs: [5494450248, 47189579]
puget_sound_north      | Selected Sites: 2 | Node IDs: [2927137481, 1315578478]
north_central          | Selected Sites: 2 | Node IDs: [50051615, 47066567]
ne_spokane             | Selected Sites: 2 | Node IDs: [47973570, 47895913]
sw_pacific_coast       | Selected Sites: 2 | Node IDs: [51408922, 49928916]
south_cascades         | Selected Sites: 2 | Node IDs: [110397230, 369930272]
se_tri_cities          | Selected Sites: 2 | Node IDs: [47965994, 48189294]
se_palouse             | Selected Sites: 0 | Node IDs: []

Total Optimal Charger Sites Selected Statewide: 14
Saved 14 optimized charger coordinates to 'optimal_ev_chargers_wa.csv'
```

[18]:
```
<folium.folium.Map at 0x7f0cc1688cd0>
```

[8]:
```
import glob
import folium
import pandas as pd
import geopandas as gpd

# ------------------------------------------------------------
# 1. Load and Filter Edge Data
# ------------------------------------------------------------
# Read and combine all regional edge files
statewide_edges = pd.concat([gpd.read_parquet(f) for f in edge_files]).
 ↪drop_duplicates(subset=["id"])

# Ensure CRS is WGS84 for Leaflet
if statewide_edges.crs != "EPSG:4326":
    statewide_edges = statewide_edges.to_crs(epsg=4326)

# Filter down to top-tier corridors to keep DOM rendering fast (<15k features)
```

```
major_types = ["motorway", "motorway_link", "trunk", "trunk_link", "primary",␣
 ↪"primary_link"]
filtered_edges = statewide_edges[statewide_edges["highway"].isin(major_types)].
 ↪copy()

print(f"Displaying {len(filtered_edges):,} major corridor edges on map...")

# ------------------------------------------------------------
# 2. Initialize Base Map
# ------------------------------------------------------------
m = folium.Map(location=[47.25, -120.85], zoom_start=7, tiles="cartodbpositron")

# ------------------------------------------------------------
# 3. Add Bounding Boxes as a FeatureGroup
# ------------------------------------------------------------
bbox_group = folium.FeatureGroup(name="8 Region Bounding Boxes", show=True)

colors = {
    "nw_olympic_peninsula": "#e41a1c",
    "puget_sound_north":    "#377eb8",
    "north_central":        "#4daf4a",
    "ne_spokane":           "#984ea3",
    "sw_pacific_coast":     "#ff7f00",
    "south_cascades":       "#a65628",
    "se_tri_cities":        "#f781bf",
    "se_palouse":           "#1b9e77"
}

for name, bbox in wa_regions_8.items():
    min_lon, min_lat, max_lon, max_lat = bbox
    bounds = [[min_lat, min_lon], [max_lat, max_lon]]
    color = colors.get(name, "#3388ff")

    folium.Rectangle(
        bounds=bounds,
        color=color,
        weight=2,
        fill=True,
        fill_color=color,
        fill_opacity=0.15,
        popup=folium.Popup(f"<b>{name}</b><br>BBox: {bbox}", max_width=250),
        tooltip=name
    ).add_to(bbox_group)

bbox_group.add_to(m)

# ------------------------------------------------------------
```

```
# 4. Add Highway Edge Network Layer
# ------------------------------------------------------------
edges_group = folium.FeatureGroup(name="Major Highway Network", show=True)

# Styling function based on highway class
def style_highway(feature):
    hw_type = feature['properties'].get('highway', '')
    if 'motorway' in hw_type:
        return {'color': '#d73027', 'weight': 3.5, 'opacity': 0.8} # Red␣
 ↪(Interstates)
    elif 'trunk' in hw_type:
        return {'color': '#f46d43', 'weight': 2.5, 'opacity': 0.8} # Orange (US␣
 ↪Highways)
    else:
        return {'color': '#2b83ba', 'weight': 1.8, 'opacity': 0.7} # Blue␣
 ↪(State Routes)

# Add GeoJson overlay
folium.GeoJson(
    filtered_edges[['highway', 'geometry']],
    style_function=style_highway,
    tooltip=folium.GeoJsonTooltip(fields=['highway'], aliases=['Type:'])
).add_to(edges_group)

edges_group.add_to(m)

# ------------------------------------------------------------
# 5. Add Layer Toggle Control
# ------------------------------------------------------------
folium.LayerControl(collapsed=False).add_to(m)

# Display Map
m
```

```
Displaying 31,667 major corridor edges on map…
```

[8]:
```
<folium.folium.Map at 0x7f26a8136d00>
```

[ ]: